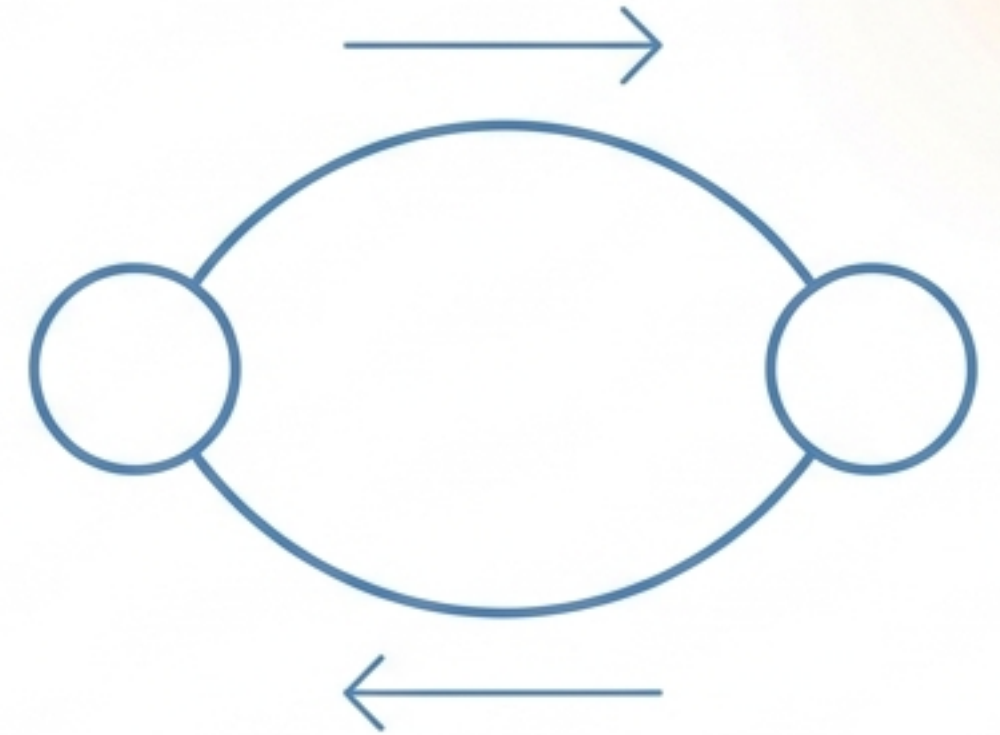


Artefactos de Tiempo Real

De peticiones estáticas a canales persistentes con WebSockets y Socket.IO.

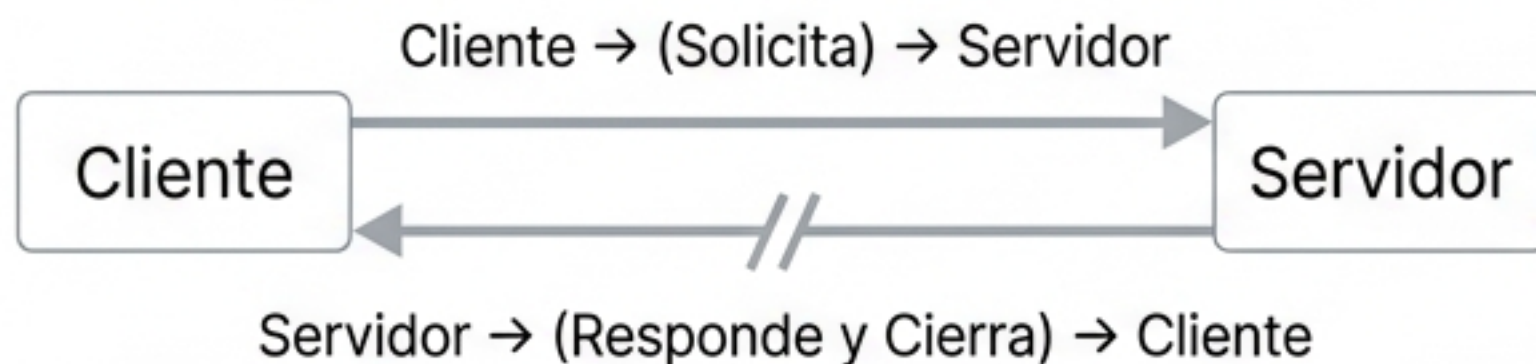


El Cambio de Paradigma

HTTP unidireccional frente a la bidireccionalidad de WebSockets.

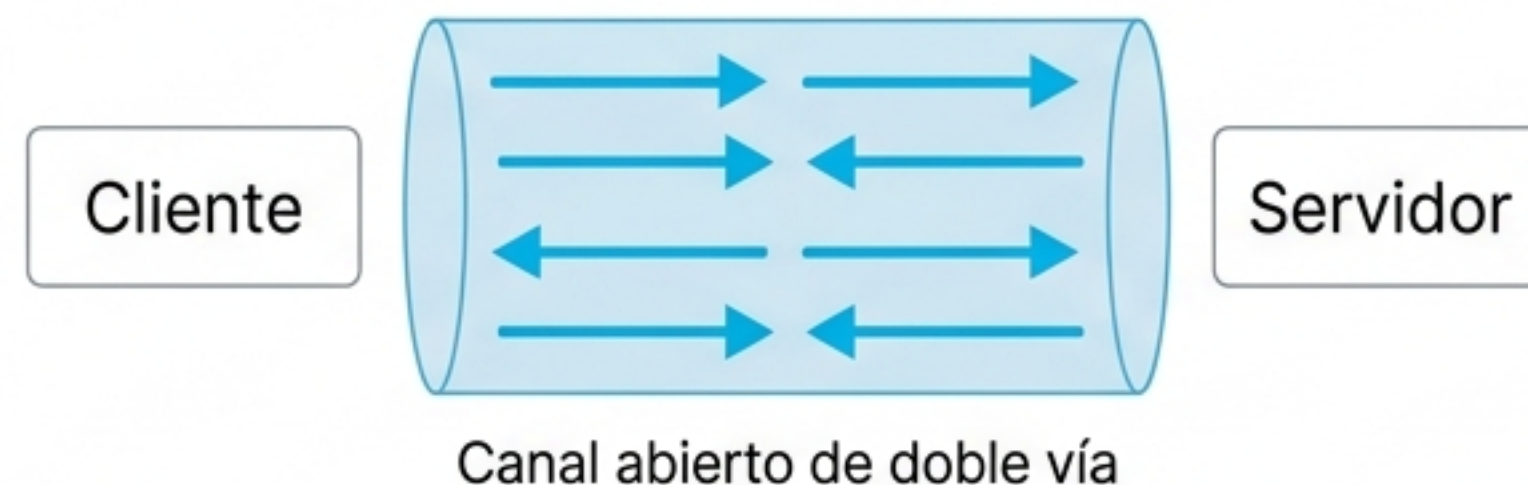
HTTP (El Walkie-Talkie)

- Protocolo unidireccional basado en solicitud-respuesta.
- El servidor no puede enviar datos sin una petición previa.



WebSockets (La Llamada Telefónica)

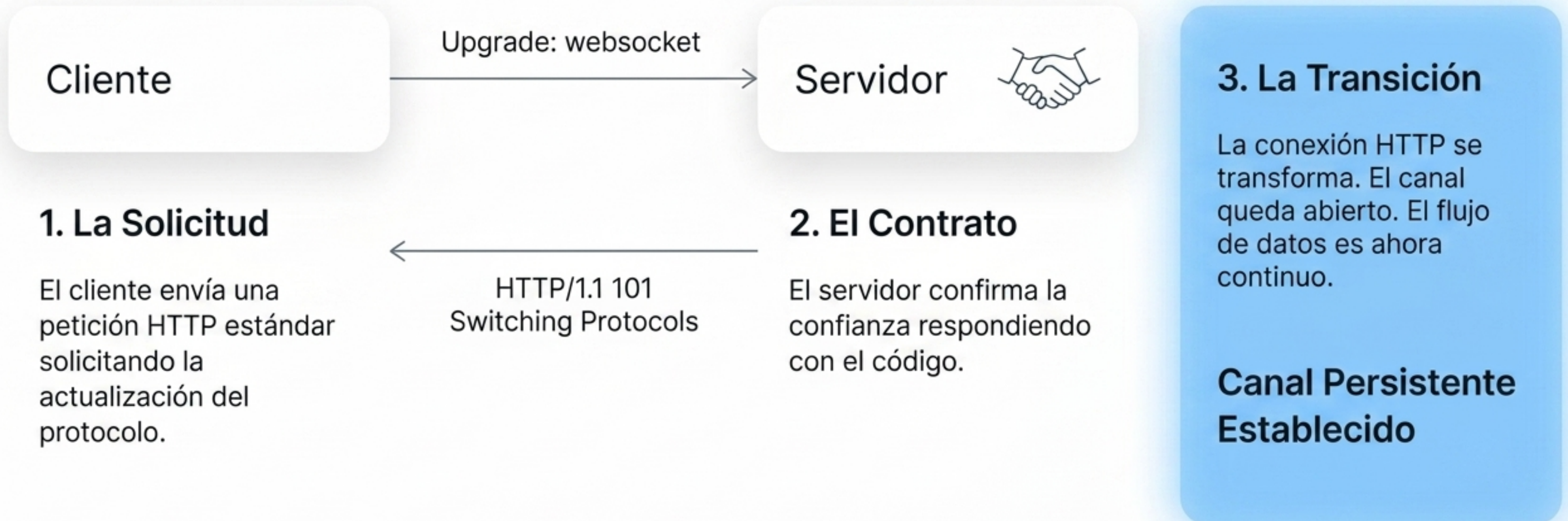
- Conexión TCP persistente y bidireccional.
- El servidor envía información en tiempo real sin requerir peticiones explícitas.



HTTP y WebSockets no son sustitutos; son complementos para ecosistemas complejos.

El Handshake

Transformando la ambigüedad de una petición en una estructura persistente.



System Capabilities

Socket.IO: Interfaz de Infraestructura

La biblioteca que dota al protocolo de capacidades resilientes.



Fiabilidad Garantizada

Establece conexiones estables atravesando proxies, balanceadores de carga, firewalls y antivirus.



Reconexión Autónoma

El agente del cliente intenta restaurar el vínculo silenciosamente si la conexión se rompe.



Detección Heartbeat

Mecanismo de pulso continuo con precisión milimétrica para detectar si el otro extremo dejó de responder.



Soporte Binario

Emisión nativa de estructuras complejas y serializables (ArrayBuffer, Blob, Buffer).

La Trinidad de las Emisiones

Control preciso sobre la dirección y el alcance de los datos.

`socket.emit`



El Privado

- **Efecto:** Envío privado a un cliente específico.

Analogía: El profesor susurra al pupitre de un alumno. Nadie más se entera.

`socket.broadcast.emit`



El Chisme

- **Efecto:** Envío a todos los clientes conectados, excepto al emisor original.

Analogía: Un alumno habla, el profesor lo repite en voz alta para el resto de la clase.

`io.emit`



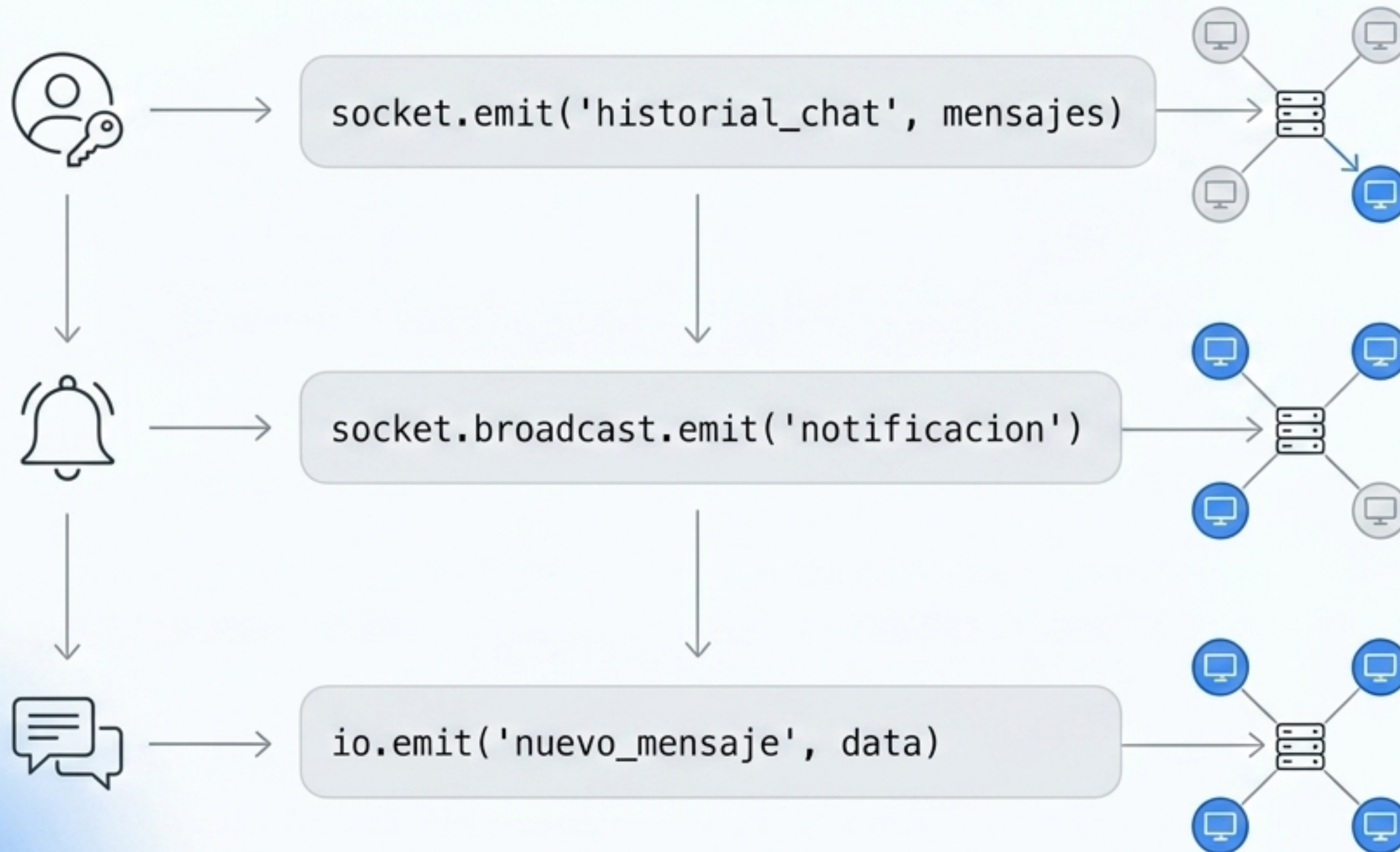
El Megáfono

- **Efecto:** Anuncio global simultáneo.

Analogía: El profesor habla con un megáfono y absolutamente todos en el aula lo escuchan.

Arquitectura en Práctica: Sala de Chat

Orquestando las tres emisiones en un flujo secuencial.



Fase 1: Contexto Individual

El servidor detecta el handshake y envía el historial guardado solo a la pantalla del nuevo cliente.

Fase 2: Conciencia Colectiva

El sistema avisa al resto de la sala que alguien llegó, sin notificar al usuario que acaba de entrar.

Fase 3: Sincronización Global

El usuario envía un mensaje. El servidor lo guarda y usa el megáfono para actualizar todas las interfaces a la vez.

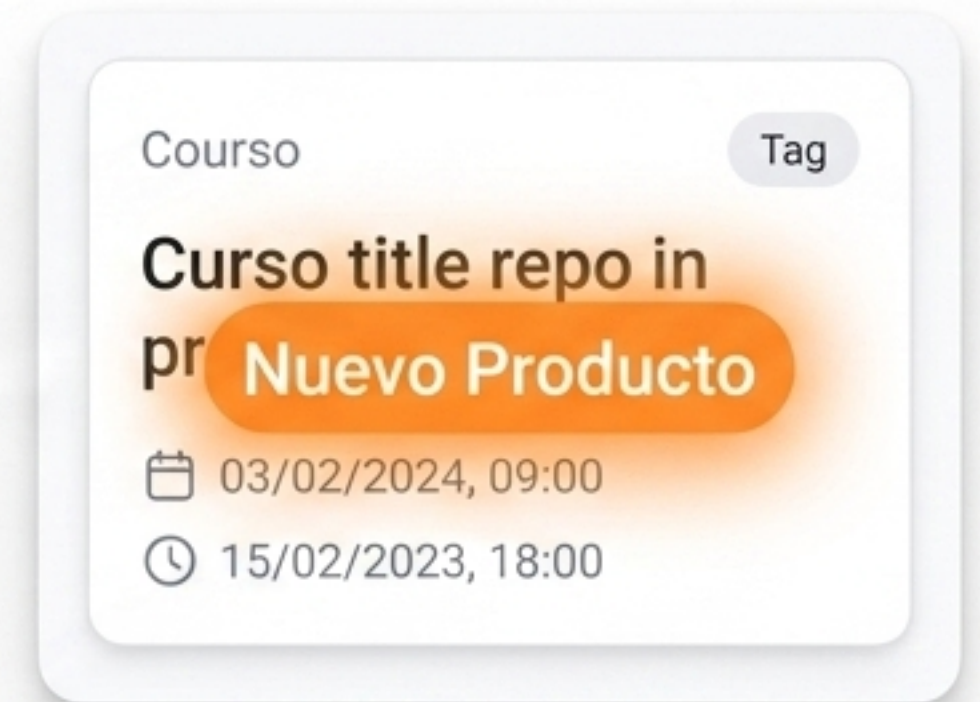
Persistencia y Tiempo Real: Catálogo

Sincronización instantánea de mutaciones de archivos (.json) en múltiples clientes.



Módulo **fs** (File System)

```
io.emit('actualizacion_productos',  
nueva_lista_json)
```



1. Carga Inicial: socket.emit lee productos.json con fs y renderiza la vista solo para ese administrador.

2. Mutación: El administrador crea un producto. El backend sobrescribe el archivo .json físicamente.

3. Propagación: io.emit dispara la nueva lista. El motor de plantillas limpia y repinta la interfaz de todos los clientes al instante.

El Ecosistema Completado

El pensamiento convertido en estructura persistente.



Sistemas que Respiran

Pasamos de un paradigma donde el cliente ruega por información, a un ecosistema donde el servidor distribuye la verdad en el instante en que ocurre.

- Con Socket.IO, manejamos la resiliencia del canal.
- Con las Tres Emisiones, controlamos la audiencia.
- Con Node.js, orquestamos la persistencia física.

El sistema ya funciona. Solo teníamos que conectarlo.